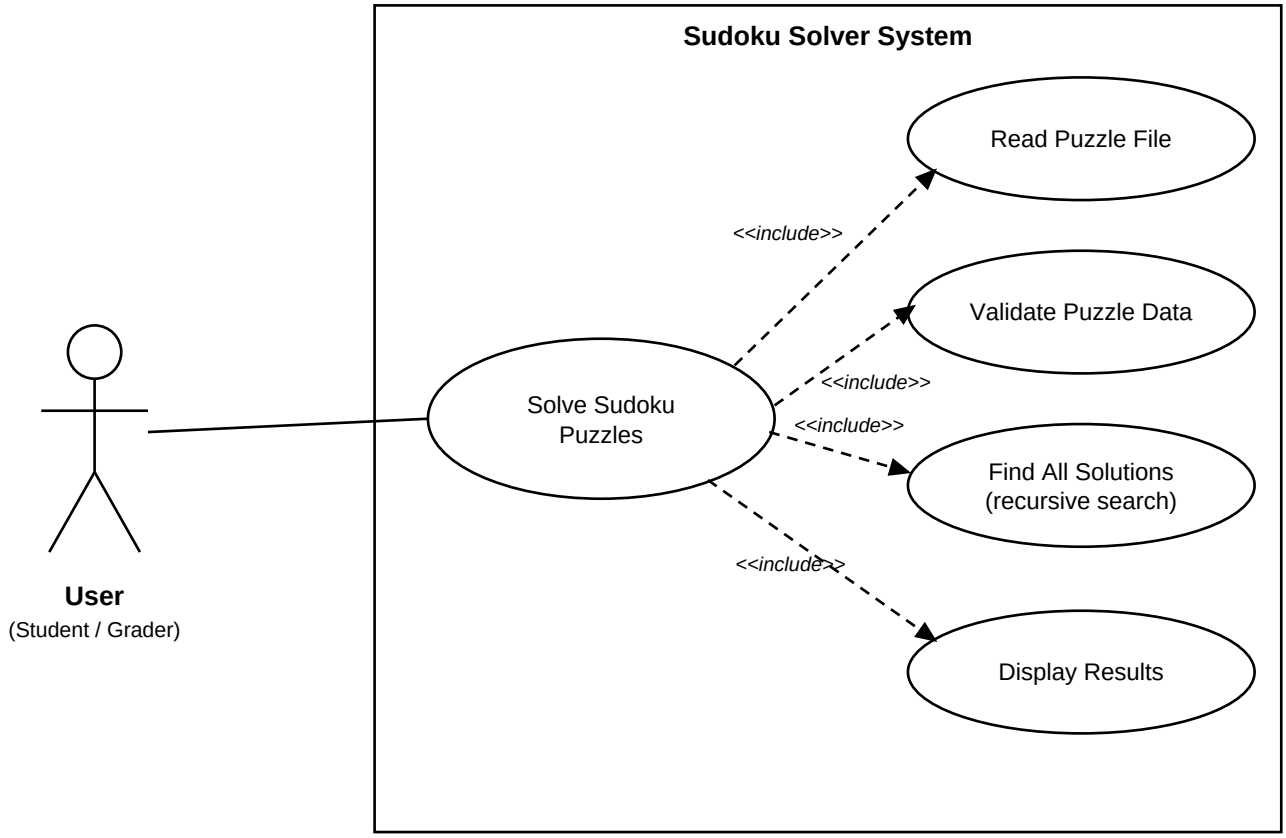
EECS 348 Assignment 4 - UML Use Case Diagram

Name: Charvi Reddy Konudula **Date:** October 15, 2026

1. Identify the use case

Use case: Solve Sudoku Puzzles. The actor is the **User** (a student running the program or a grader). The system is the recursive, object-oriented Sudoku Solver program (Assignment4.cpp).



2. Brief description

The user runs the program. For each of the five puzzle files (puzzle1.txt through puzzle5.txt) the system reads the puzzle, checks that it is valid, uses a recursive depth-first search with backtracking to find every solution, and then displays the puzzle, each solution found, and the number of solutions (or "No solution found").

3. Step-by-step flow of events (outline)

1. The user runs Assignment4.exe.
2. The system starts with puzzle1.txt and prints the file name.
3. The system reads the 81 cells from the file (digits 1-9 and _ for blanks).
4. The system validates the data (file opens, 81 cells, only _ or 1-9).
5. The system prints the puzzle exactly as it was read.

6. The system finds the first empty cell and tries each candidate digit 1-9 that does not break a row, column, or 3x3 block rule.
7. The system calls itself recursively for the next cell; if no digit works, it backtracks (erases the digit and tries the next one).
8. When all 81 cells are filled, the system counts and prints that solution, then keeps searching for more solutions.
9. When the search ends, the system prints the total number of solutions, or "No solution found" if there were none.
10. The system repeats steps 2-9 for puzzle2.txt through puzzle5.txt, then ends.

4. Detailed description

Use case name	Solve Sudoku Puzzles
Actor	User (student or grader)
Goal	Find and display every solution for each of the five Sudoku puzzle files.
Trigger	The user runs Assignment4.exe.
Preconditions	Assignment4.exe and puzzle1.txt through puzzle5.txt are in the same folder. Each puzzle file holds 9 rows of 9 space-separated symbols, using 1-9 for clues and _ for blanks.
Postconditions (success)	For every puzzle the screen shows the file name, the puzzle as read, all solutions found (numbered) with a total, or "No solution found". The puzzle files are not changed.
Main flow	Steps 1-10 in section 3.
Alternative flow A: multiple solutions	If a puzzle has more than one solution (for example puzzle5.txt has two), the system keeps searching after the first solution and prints each one in turn, followed by the total.
Alternative flow B: no solution	If every candidate digit fails (for example puzzle4.txt), the search ends with no complete grid and the system prints "No solution found".
Exception: file cannot be opened	The system prints an error naming the file and moves on to the next puzzle.
Exception: invalid data	If a file has fewer than 81 cells or a symbol other than _ or 1-9, the system prints an error describing the problem and moves on to the next puzzle.
Included use cases	Read Puzzle File; Validate Puzzle Data; Find All Solutions (recursive depth-first search with backtracking); Display Results.
Rules and constraints	Each digit 1-9 appears exactly once in every row, every column, and every 3x3 block. The search must be recursive and object-oriented (a SudokuSolver class).
Frequency of use	Once per run of the program; the use case repeats for each of the five puzzle files.